

Reviewer Pack — Algebraic Automorphic Forms and DPLL Refutation Tree Width

Entry 1c07529f0c6c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 14:56:58 UTC

Algebraic Automorphic Forms and DPLL Refutation Tree Width

Entry ID: 1c07529f0c6c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 14:56:58 UTC

1. Conjecture

Field A (mathematical branch): Automorphic Forms Theory **Field B** (complexity object): Complexity Theory: DPLL Refutation Complexity

Statement:

[‘For every instance of a 3-CNF formula with n variables, the number of distinct algebraic automorphic forms associated with its satisfying assignments is upper-bounded by a function $f(n)$ that grows exponentially with n .’, ‘If an instance has a small DPLL refutation tree width, then there are significantly fewer algebraic automorphic forms associated with its satisfying assignments than in instances with larger DPLL refutation tree widths.’, ‘An instance requires a large number of algebraic automorphic forms to represent all its satisfying assignments if and only if it is hard for the DPLL algorithm to find a refutation.’]

Rationale (proposer’s reasoning):

[‘Automorphic forms provide a rich algebraic structure that can capture complex patterns in data. By linking this structure with the complexity of satisfiability problems, we may uncover new insights into the computational hardness of such problems.’, ‘The use of automorphic forms could potentially reveal hidden structures in Boolean functions, which are related to the refutation tree width and thus to the computational complexity.’, ‘This conjecture could offer a novel perspective on proving lower bounds for NP-completeness by leveraging algebraic geometry.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 03dbc52c08a47174

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The number of distinct algebraic automorphic forms associated with satisfying assignments in a 3-CNF formula is upper-bounded by an exponential function $f(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	0.95	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] != -clause[j] for i in range(n) for j in range(i + 1, n)):
                clauses.append(clause)
        return clauses

    def dpll_refutation_tree_width(clauses):
        # Simplified DPLL algorithm to estimate tree width
        stack = []
        literals = set()
        for clause in clauses:
            literals.update(abs(lit) for lit in clause)
        while literals:
            literal = random.choice(list(literals))
            literals.remove(abs(literal))
            stack.append((literal, []))
            while stack:
                (lit, path) = stack.pop()
                if not any(lit in cl for cl in clauses):
                    return len(path)
                new_clauses = [cl for cl in clauses if lit not in cl and -lit not in cl]
                new_literals = set(abs(lit) for lit in new_clauses[0]) if new_clauses else set()
                stack.append((random.choice(list(new_literals)), path + [(lit, new_clauses)]))
        return 0
```

```

def algebraic_automorphic_forms(clauses):
    # Simplified mapping to count automorphic forms
    forms = set()
    for clause in clauses:
        form = tuple(sorted(abs(lit) for lit in clause))
        forms.add(form)
    return len(forms)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    clauses = generate_3cnf(n)
    width = dpll_refutation_tree_width(clauses)
    forms = algebraic_automorphic_forms(clauses)
    results.append((n, width, forms))

metric_value = sum(forms for _, _, forms in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(width < forms for _, width, forms in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "algebraic_automorphic_forms",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d110a0fc.py", line 83, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d110a0fc.py", line 60, in run_trial
    width = dpll_refutation_tree_width(clauses)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d110a0fc.py", line 45, in dpll_refutati
    stack.append((random.choice(list(new_literals)), path + [(lit, new_clauses)]))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
```

IndexError: Cannot choose from an empty sequence

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity according to the pre-registered support and falsification conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12291
2	preregistration	ollama_remote	glm4:latest	0	0	5569
3	novelty	ollama_remote	glm4:latest	0	0	4918
4	novelty	ollama_remote	glm4:latest	0	0	5187
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13658
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7699
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10657
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9683
9	judge	ollama_remote	glm4:latest	0	0	8123

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 77786 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1c07529f0c6c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1c07529f0c6c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1c07529f0c6c.tar.gz` (if generated)